

Applied AI and Engineering AI System

Learning Resources

Learning Objectives (Bloom's Taxonomy)

By the end of this module, all learners will be able to:

1. **Develop** multi-provider AI applications using unified routers (LiteLLM), structured output schemas (Pydantic/Instructor), and agentic workflows (LangGraph/LangChain). *(Bloom's: Apply)*
2. **Implement** multi-tier caching (Prompt & Semantic Caching), security guardrails, and full-stack tracing (Langfuse) to protect and optimize live AI endpoints. *(Bloom's: Apply)*
3. **Deploy** deep learning models using industrial serving engines (**vLLM, Triton, TorchServe**) with gRPC/REST APIs and GPU resource scheduling. *(Bloom's: Apply)*
4. **Compile** dynamic PyTorch models into optimized ONNX and NVIDIA TensorRT engines with multi-precision quantization (**FP16, BF16, INT8, FP8**). *(Bloom's: Analyze)*
5. **Configure** continuous batching, KV caching, and speculative decoding to maximize hardware throughput and minimize generation latency. *(Bloom's: Apply)*
6. **Profile** distributed AI architectures using PyTorch DDP across multi-GPU setups while monitoring system health, TTFT, and ITL metrics. *(Bloom's: Evaluate)*

Day 1: AI Building Blocks & Model Integration

Focus: Mastering the AI stack, vendor abstractions, and building reliable, typed AI applications.

- **Core Concepts:**

- **Vendor SDK Architectures:** Understanding trade-offs between direct API integration and the friction of managing distinct client classes vs. unified gateways.
- **Unified Model Gateways:** Implementing LiteLLM and OpenRouter to standardize schemas, manage rate-limit failovers, and optimize routing.
- **Structured Outputs:** Enforcing runtime contracts using Pydantic and Instructor to validate JSON fields and handle schema auto-healing.
- **Prompt Engineering Systems:** Implementing System Prompt Framing, Few-Shot examples, and Chain-of-Thought (CoT) for complex task decomposition.

Resources:

- [Claude Learning Path](#)
- [Prompt Engineering Guide](#)

- [Gen AI for Beginners - Microsoft](#)

Day 2-3: RAG Pipelines & Advanced Retrieval

Focus: Building context-aware systems, embedding physics, and retrieval optimization.

- **Core Concepts:**
 - **Context Window Engineering:** Managing token relevance, addressing the "Lost-in-the-Middle" effect, and applying dynamic context assembly based on user intent.
 - **Embedding Models & Vector Physics:** Understanding dense vector spaces (1536d+), distance metrics (Cosine vs. Dot Product), and Matryoshka Representations (MRL) for storage efficiency.
 - **Ingestion & Chunking Mechanics:** Moving beyond naive character splitting to layout-aware parsing and semantic boundary chunking.
 - **Hybrid Search & Re-ranking:** Implementing RRF (Reciprocal Rank Fusion) combining Sparse (BM25) and Dense vectors, followed by two-stage cross-encoder re-ranking.
- **Resources:**
 - **NVIDIA: Augment Your LLM Using Retrieval-Augmented Generation**
<https://lnkd.in/gFSaVuXA>
 - **Stanford CS25 Lecture: Retrieval-Augmented Language Models**
<https://lnkd.in/gTtpSPs>
 - <https://www.meilisearch.com/blog/hybrid-search-rag>
 - <https://www.pinecone.io/learn/series/rag/rerankers/>

Day 4-6: Production Architecture & Industrial Serving

Focus: Deploying models at scale in production environments.

- **Core Concepts:**
 - **Common System Architectures:** Decoupling services using Microservices (REST/gRPC) and Event-Driven architectures to manage traffic spikes.
 - **High Availability Systems:** Designing resilient AI pipelines with redundancy, failover mechanisms, load balancing, and fault-tolerant components.
 - **Industrial Inference Engines:** Replacing naive wrappers with vLLM (PagedAttention), NVIDIA Triton (Multi-backend), and TorchServe to increase request throughput.
- **Resources:**
 - **MLSysBook — Machine Learning Systems** — AI/ML system architecture, production ML systems, inference, scalability, and deployment
[MLSysBook](#) (RECOMMENDED)
 - **MLSysBook — Inference at Scale** — Production inference architecture, online/batch inference, latency, throughput, batching, GPU utilization, and scaling
[Inference at Scale](#)
 - **Made With ML — MLOps** — End-to-end transition from trained models to

production services, deployment, testing, monitoring, and serving

[Made With ML MLOps](#)

- **Full Stack Deep Learning** — Practical course covering the full lifecycle of deep-learning applications, including deployment, infrastructure, monitoring, and production systems
[Full Stack Deep Learning](#)
- **vLLM Documentation** — High-throughput LLM serving, continuous batching, KV-cache management, and production inference
[vLLM](#)

Day 7: Graph Compilation & Model Optimization

Focus: Maximizing hardware throughput and minimizing latency.

- **Core Concepts:**

- **AOT Compilation:** Using ONNX and NVIDIA TensorRT for graph fusion (kernel fusion) and decreasing execution latency.
- **Quantization Strategies:** Implementing PTQ vs. QAT, and leveraging AWQ/GPTQ and FP8 for VRAM reduction.
- **High-Throughput Generation:** Implementing Continuous Batching, Speculative Decoding, and Chunked Prefill to maximize concurrent user capacity.

- **Resources:**

- **NVIDIA Triton Inference Server Documentation** — Industrial model serving, dynamic batching, model repositories, HTTP/gRPC serving, and performance optimization
[NVIDIA Triton Inference Server](#)
- **ONNX Runtime Documentation** — Model conversion, optimized inference runtimes, hardware acceleration, and production deployment
[ONNX Runtime](#)
- **Hugging Face — Quantization** — Practical overview of GPTQ, AWQ, bitsandbytes, Quanto, and other modern quantization approaches.
[Hugging Face Quantization Guide](#)
- **LLM.int8() Paper** — Foundation for mixed-precision 8-bit LLM inference and outlier-aware quantization.
[LLM.int8\(\): 8-bit Matrix Multiplication for Transformers at Scale](#)
- **GPTQ Paper** — Important post-training quantization (PTQ) method for compressing large language models to 3–4 bits.
[GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers](#)
- **AWQ Paper** — Activation-aware weight quantization, widely used for efficient LLM inference.
[AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration](#)
- **NVIDIA TensorRT-LLM — Quantization** — Practical production-oriented

coverage of FP16, BF16, INT8, FP8, INT4 and quantized LLM inference.

[TensorRT-LLM Documentation](#)

- **bitsandbytes — Quantization** — Hands-on resource for 8-bit and 4-bit model loading and inference with Transformers.

[bitsandbytes Documentation](#)

Day 8: Distributed Systems, Orchestration & Cloud Infrastructure

Focus: Scaling infrastructure, distributed compute, and container orchestration.

- **Core Concepts:**

- **Introduction to Kubernetes (K8s):** Understanding the role of container orchestration for AI workloads, including Pod management, Deployments, and Services.
- **Distributed Data Parallelism (DDP):** Replicating models across GPUs and synchronizing gradients via AllReduce.

- **Resources:**

- **PyTorch — Distributed Training Overview** — Broader introduction to distributed training concepts and PyTorch's distributed APIs.
[PyTorch Distributed Overview](#)
- **Kubernetes — Basics** — Official hands-on introduction covering Pods, Deployments, Services, scaling, and application management.
[Kubernetes Basics](#)